

# PRAM (Parallel Random Access Machine) - Detailed Study Guide

---

This guide explains the foundational theoretical model used to design and analyze parallel algorithms before they are implemented in code (like Pthreads or OpenMP). The PRAM model helps us answer: "*If I had infinite processors, how fast could I solve this problem?*"

---

## 1. What is the PRAM Model?

The PRAM model is a theoretical machine consisting of:

1. An unbounded (infinite) number of processors,  $P_1, P_2, P_3, \dots$
2. An unbounded (infinite) shared memory.
3. A common clock (all processors execute instructions strictly in synchronized steps).

**How it is utilized:** We use PRAM to ignore hardware limitations (like cache misses, network latency, or having only 8 cores) so we can discover the absolute mathematical limit of parallelizing an algorithm.

### The Four Memory Access Models

Because all processors share the same memory, what happens if two processors try to access the same memory address simultaneously? We define four STRICT rulesets:

1. **EREW (Exclusive Read, Exclusive Write):** The strictest model. No two processors can read OR write to the same address at the same time.
2. **CREW (Concurrent Read, Exclusive Write):** Multiple processors can read the same address simultaneously, but only one can write. (*Used most often in algorithm design*).
3. **ERCW (Exclusive Read, Concurrent Write):** Rarely used because it makes no logical sense.
4. **CRCW (Concurrent Read, Concurrent Write):** The most powerful model. Multiple processors can read AND write to the same address simultaneously.

**Resolving CRCW Write Conflicts:** If 5 processors try to write to cell `A[0]` in CRCW, who wins?

- **Common CRCW:** Only allowed if all processors are trying to write the *exact same value*.
  - **Arbitrary CRCW:** A randomly selected processor succeeds; the others fail.
  - **Priority CRCW:** The processor with the lowest ID (e.g.,  $P_1$  beats  $P_5$ ) wins.
- 

## 2. The Work-Time Framework (Performance Metrics)

When analyzing a PRAM algorithm, you must calculate these metrics:

1. **Time / Step Complexity ( $T_p$ ):** The number of parallel time steps required using  $P$  processors. E.g., Tree-based sum takes  $O(\log n)$  time.
2. **Work Complexity ( $W$ ):** The total number of operations performed by all processors combined. Usually,  $W = T_1$  (the time it takes a sequential algorithm).

- 3. **Cost:**  $\text{Cost} = T_p \times P$ . A parallel algorithm is **Cost-Optimal** if its Cost equals the best known sequential time  $O(n)$ .
- 4. **Speedup (\$\$S\$):**  $S = T_1 / T_p$ . How much faster it is compared to the sequential version.
- 5. **Efficiency (\$\$E\$):**  $E = S / P$ . How well you are utilizing the processors. Ideal efficiency is 1.0 (or 100%).

### 3. How to Utilize PRAM to Design Algorithms

Let's look at how we utilize PRAM to build parallel logic step-by-step.

#### Classic PRAM Algorithm: Finding the Max in an Array (Tree Reduction)

**Goal:** Find the maximum element in an array of size  $N$  using  $N/2$  processors. **Model:** EREW or CREW.  
**Strategy:** Binary Tree Reduction.

Array: [3, 1, 9, 2, 7, 5, 8, 4] (N=8)  
Step 1: P1 compares(3, 1) -> 3. P2 compares(9, 2) -> 9. P3(7,5)->7. P4(8,4)->8.  
Array state conceptually: [3, 9, 7, 8]  
Step 2: P1 compares(3, 9) -> 9. P2 compares(7, 8) -> 8.  
Array state conceptually: [9, 8]  
Step 3: P1 compares(9, 8) -> 9.  
Final Answer: 9.

- **Time Complexity:**  $O(\log N)$  (Because the array halves every step).
- **Processors Used:**  $O(N)$ .
- **Work Complexity:**  $O(N)$ .

#### The CRCW "Magic" Algorithm: Finding Max in $O(1)$ Time!

CRCW is so theoretically powerful that it can achieve the impossible. **Goal:** Find the max of array  $A$  of size  $N$  in exactly 1 step (Constant Time  $O(1)$ ). **Processors:**  $O(N^2)$  processors. **Strategy:**

- 1. Have a boolean array `win[N]` initialized to 1 (True).
- 2. Assign Processor  $P_{i,j}$  to compare  $A[i]$  and  $A[j]$ .
- 3. If  $A[i] < A[j]$ , Processor  $P_{i,j}$  writes 0 to `win[i]`. (Multiple processors might try to write 0 simultaneously, which is completely legal in Common CRCW!).
- 4. After 1 step, only the absolute maximum element will still have 1 in its `win` slot, because it was never smaller than anything else!

### 4. Exam Cheat Sheet: Adapting PRAM Concepts

| If the question asks...           | How to answer / What to use...                                                                                                 |
|-----------------------------------|--------------------------------------------------------------------------------------------------------------------------------|
| "Is this algorithm Cost-Optimal?" | Calculate $\text{Cost} = \text{Time} \times \text{Processors}$ . If $\text{Cost} == O(n)$ (same as sequential time), then YES. |

| If the question asks...                           | How to answer / What to use...                                                                                                                                          |
|---------------------------------------------------|-------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| "Design a minimum/maximum finder in $O(\log N)$ " | Use a <b>Binary Tree Reduction</b> under the EREW or CREW model using $N/2$ processors.                                                                                 |
| "Design an algorithm that runs in $O(1)$ time"    | You absolutely <b>must</b> use the <b>CRCW</b> model and throw $O(N^2)$ processors at it to do all-to-all comparisons simultaneously.                                   |
| "Explain why EREW is weaker than CREW"            | In EREW, if 100 processors need the same variable, you must broadcast it in a tree, taking $O(\log 100)$ steps. In CREW, they all read it on step 1 taking $O(1)$ time. |
| "Compute the speedup / efficiency"                | $\text{Speedup} = \frac{T_{\text{sequential}}}{T_{\text{parallel}}}$ . $\text{Efficiency} = \frac{\text{Speedup}}{\text{Processors}}$ .                                 |

## 5. Multi-Layer Exam Preparation: Solved Mock Questions

### Question A: Broadcasting Data in EREW

**Problem Statement:** You have a single scalar variable  $x$  stored at memory index 0. You have an EREW PRAM with 8 processors. All 8 processors need a copy of  $x$ .

1. Can they all read it on step 1? Why or why not?
2. Design a step-by-step PRAM strategy to distribute the data as fast as possible. What is the time complexity?

#### Step-by-step Solution:

1. **Why they can't:** In the EREW (Exclusive Read) model, it is illegal for multiple processors to read memory index 0 simultaneously. It would cause a hardware fault in this theoretical machine.
2. **The Strategy (Binary Broadcast/Doubling):**
  - Step 1:  $P_1$  reads  $x$  from index 0 and writes it to index 1.
  - Step 2:  $P_1$  and  $P_2$  read from 0 and 1, and write to indices 2 and 3.
  - Step 3: Processors  $P_1 \dots P_4$  read indices  $0 \dots 3$  and write to indices  $4 \dots 7$ .
  - Now 8 cells have the data.
3. **Time Complexity:** Because the number of copies doubles every step ( $1 \rightarrow 2 \rightarrow 4 \rightarrow 8$ ), the time complexity is  $O(\log N)$ .

### Question B: Cost-Optimality Analysis

**Problem Statement:** To sum an array of size  $N$ , a student proposes an algorithm that uses  $N/\log N$  processors. The algorithm completes the sum in  $O(\log N)$  parallel steps.

1. What is the sequential time complexity to sum an array of size  $N$ ?
2. What is the Speedup of the student's algorithm?
3. Is the student's algorithm cost-optimal?

#### Step-by-step Solution:

1. **Sequential Time:** A standard `for` loop summing  $N$  elements takes  $O(N)$  time.  $T_1 = O(N)$ .
2. **Speedup Calculation:**  $S = T_1 / T_p$ .  $S = O(N) / O(\log N)$ .
3. **Cost-Optimality Check:**  $\text{Cost} = \text{Processors} \times \text{Parallel\_Time}$ .  $\text{Cost} = (N / \log N) \times \log N$ . The  $\log N$  terms cancel out.  $\text{Cost} = O(N)$ . *Conclusion:* Yes, the algorithm is perfectly cost-optimal because its cost  $O(N)$  equals the best known sequential time  $O(N)$ .

### Question C: Point Jumping / List Ranking

**Problem Statement:** You have a Linked List of size  $N$  stored randomly in memory. You want every node to find a pointer directly to the absolute end/tail of the list. A sequential loop tracing pointers takes  $O(N)$  time. Explain the PRAM "Pointer Jumping" algorithm to do this in  $O(\log n)$  time.

#### Step-by-step Solution:

1. **Concept:** Instead of following pointers one by one, every node looks at its single "next" pointer, and replaces it with its "next's next" pointer concurrently.
2. **Execution steps:**
  - Initialize: Assign 1 processor to every single node.
  - Step 1: All nodes  $i$  simultaneously do: `next[i] = next[next[i]]`. (Now everyone points 2 jumps ahead).
  - Step 2: Do it again. `next[i] = next[next[i]]`. (Now everyone points 4 jumps ahead).
  - Step  $k$ : Everyone points  $2^k$  jumps ahead.
3. **Result:** Because the jump distance doubles unconditionally every step, after exactly  $\log_2(N)$  steps, every single node in the list will be pointing directly at the tail marked as `NULL`. Time complexity is  $O(\log N)$ .